



Count Unique Subsequences of Length N-2

Company: TCS

Round: Offline

Year: 2022

Difficulty: Medium - Hard

Topic: String, Dynamic Programming, Hashing

Source: <https://www.designqa.in/10965/tcs-nqt-coding-questions-%26-answers-2022-set-41>

Problem Description

Given a string S of length N, find the **number of unique subsequences** that can be formed from S, where each subsequence must have exactly **N - 2 characters**.

A subsequence is obtained by **deleting exactly two characters** from the original string while maintaining the order of the remaining characters.

The string contains only digits from **0 to 9**.

Since the answer can be very large, return the result modulo:

$10^9 + 7$

Input Format

- The first line contains an integer N, representing the length of the string.
- The second line contains the string S.
- The string contains only digits from 0 to 9.
- We need to find unique subsequences of length $N - 2$

Output Format

Print a single integer representing the number of unique subsequences of length $N - 2$ that can be formed from the given string.

The result must be printed modulo $10^9 + 7$.

Sample Input

4
1234

Sample Output

6

Explanation

Given:

$N = 4$

$S = "1234"$

We need to find subsequences of length:

$N - 2 = 4 - 2 = 2$

So, we have to **remove exactly 2 characters** from the string while keeping the order of the remaining characters.

The possible unique subsequences are:

12

13
14
23
24
34

There are **6 unique subsequences**.

Therefore:

Output = 6

In simple terms, **choose any 2 characters from the 4-character string while maintaining their original order**. Since all characters are different, all 6 possible pairs are unique.

Approach to Solve This Problem:

- We need subsequences of length $N - 2$, so exactly 2 characters must be removed from the string.
- Generate all possible ways of removing 2 positions from the string.
- For each choice, keep the remaining characters in their original order to form a subsequence.
- Use a HashSet to store the subsequences, so duplicate subsequences are counted only once.
- After checking all possibilities, the size of the HashSet gives the number of unique subsequences.
- Return the answer modulo $10^9 + 7$.

Java Solution:

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         String s = sc.next();
7         HashSet<String> set = new HashSet<>();
8         for (int i = 0; i < n; i++) {
9             for (int j = i + 1; j < n; j++) {
10                 StringBuilder sub = new StringBuilder();
11
12                 for (int k = 0; k < n; k++) {
13                     if (k != i && k != j) {
14                         sub.append(s.charAt(k));
15                     }
16                 }
17                 set.add(sub.toString());
18             }
19         }
20         long MOD = 1000000007;
21         System.out.println(set.size() % MOD);
22         sc.close();
23     }
24 }
```

Solution:

The solution is based on the fact that a subsequence of length $n - 2$ is obtained by **removing exactly two characters** from the original string.

1. Consider every possible pair of positions (i, j) to remove.
2. Create a new string using all characters except the characters at positions i and j .
3. Store the resulting subsequence in a **HashSet**.
4. If the same subsequence is produced more than once, the HashSet automatically keeps only one copy.
5. After considering all pairs, the **size of the HashSet** gives the number of unique subsequences.
6. Return the result modulo $10^9 + 7$.

Complexity Analysis:

Time Complexity: $O(N^3)$

There are $O(N^2)$ pairs of characters to remove. For each pair, we traverse the entire string of length N to create the subsequence.

Space Complexity: $O(N^3)$ in the worst case

There can be up to $O(N^2)$ unique subsequences, and each subsequence has length $N-2$, so storing them in the HashSet can require $O(N^3)$ total space.